Projet d'Analyse Numérique

Interpolation et Intégration Numérique pour l'Analyse de Données Expérimentales

Travail demandé

Table des matières

Présentation générale **2**

1 Problématique et objectifs **3**
1.1 Problématique . . . 3
1.1.1 Problème 1 : Refroidissement d'un composant électronique . . . 3
1.1.2 Problème 2 : Écoulement dans un canal . . . 3
1.2 Objectifs du projet . . . 3

2 Travail théorique (à rédiger dans le rapport) **4**
2.1 Interpolation polynomiale . . . 4
2.2 Interpolation par splines . . . 4
2.3 Intégration numérique . . . 4

3 Travail pratique (à coder) **5**
3.1 Interpolation polynomiale . . . 5
3.2 Splines . . . 5
3.3 Méthodes d'intégration . . . 5
3.4 Applications . . . 5
3.5 Visualisation . . . 6
3.6 Programme principal . . . 6

4 Travail d'analyse (à rédiger dans le rapport) **7**
4.1 Analyse de l'interpolation . . . 7
4.2 Analyse de l'intégration . . . 7
4.3 Problème inverse . . . 7

5 Livrables attendus **8**
5.1 Code source . . . 8
5.2 Rapport . . . 8
5.3 Soutenance . . . 8

6 Annexes **9**
6.1 Annexe A : Données complémentaires . . . 9
6.1.1 A.1 Fonction de Runge . . . 9
6.1.2 A.2 Points de Tchebychev . . . 9
6.1.3 A.3 Fonction test pour la convergence . . . 9

6.2 Annexe B : Données expérimentales 9

Présentation générale

Ce document décrit l'intégralité du travail à réaliser pour ce mini-projet. Il est structuré en sections correspondant aux différentes étapes. Chaque section précise les objectifs, les résultats attendus et ce que vous devez coder vous-mêmes.

1 Problématique et objectifs

1.1 Problématique

Dans de nombreuses applications industrielles et scientifiques, on dispose de mesures effectuées à des instants discrets, mais on a besoin de connaître le comportement continu d'un système pour calculer des grandeurs physiques intégrales.

1.1.1 Problème 1 : Refroidissement d'un composant électronique

Un capteur de température enregistre les mesures suivantes :

Temps t (s)	0	1	2	3	4	5	6	7	8	9	10
Température T (°C)	90	85	72	63	58	52	48	45	43	41	40

Questions :

1. Quelle est la température à $t = 2,5$ s ? À $t = 7,3$ s ?
2. Quelle quantité de chaleur a été dissipée entre $t = 0$ et $t = 10$ s sachant que :

$$Q = \int_0^{10} h \cdot (T(t) - T_{\text{amb}})\, dt$$

avec $h = 50$ J/°C et $T_{\text{amb}} = 20$ °C ?

1.1.2 Problème 2 : Écoulement dans un canal

Mesures de vitesse à différentes positions :

Position x (m)	0	0,5	1,2	1,8	2,5	3,1	3,7	4,2	4,8	5,3	6,0
Vitesse v (m/s)	0	2,1	3,8	5,2	6,4	7,0	7,3	7,2	6,8	5,9	4,5

La largeur du canal varie selon $w(x) = 0,5 + 0,1x$ m.
Question : Calculer le débit volumique total :

$$D = \int_0^6 v(x) \cdot w(x)\, dx$$

1.2 Objectifs du projet

Objectif principal : Implémenter et analyser des méthodes numériques d'interpolation et d'intégration pour résoudre des problèmes concrets.
Objectifs spécifiques :
— Implémenter les méthodes d'interpolation polynomiale (Lagrange et Newton)
— Implémenter les splines linéaires et cubiques
— Implémenter les méthodes d'intégration numérique (rectangle, trapèzes, Simpson)
— Implémenter une méthode d'intégration adaptative
— Appliquer ces méthodes aux problèmes de refroidissement et d'écoulement
— Analyser la précision et la stabilité des méthodes
— Résoudre un problème inverse d'estimation de paramètre

2 Travail théorique (à rédiger dans le rapport)

2.1 Interpolation polynomiale

À faire : Rédiger une synthèse théorique comprenant :

1. **Formule de Lagrange** : écrire la formule générale, expliquer le rôle des polynômes $L_i(x)$.
2. **Méthode de Newton** : écrire la forme de Newton, définir les différences divisées, donner l'algorithme de calcul des coefficients.
3. **Phénomène de Runge** : décrire le phénomène, donner l'exemple classique $f(x) = \dfrac{1}{1+25x^2}$, expliquer pourquoi il se produit, proposer une solution.

2.2 Interpolation par splines

À faire : Rédiger une synthèse théorique comprenant :

1. **Spline linéaire** : définition sur chaque intervalle, propriétés.
2. **Spline cubique naturelle** : définition, conditions, système d'équations, conditions aux limites naturelles $S''(a) = S''(b) = 0$.

2.3 Intégration numérique

À faire : Rédiger une synthèse théorique comprenant :

1. **Méthode du rectangle** : formule, ordre, expression de l'erreur.
2. **Méthode des trapèzes** : formule simple et composée, ordre, expression de l'erreur.
3. **Méthode de Simpson** : formule simple et composée, ordre, condition sur le nombre d'intervalles.
4. **Méthode adaptative** : principe, estimation de l'erreur, algorithme de Simpson adaptatif.

3 Travail pratique (à coder)

3.1 Interpolation polynomiale

À faire : Implémenter la classe `PolynomialInterpolation` avec les méthodes suivantes :

- `__init__(self, x_points, y_points)` : initialise l'interpolateur, vérifie que les points sont distincts.
- `lagrange(self, x_eval)` : évalue le polynôme de Lagrange.
- `newton_coefficients(self)` : calcule les coefficients par différences divisées.
- `newton_eval(self, x_eval)` : évalue le polynôme de Newton.
- `evaluate(self, x_eval, method='newton')` : interface unifiée.

Contraintes :

- Gérer les erreurs (points non distincts, longueurs différentes).
- Utiliser NumPy pour les calculs vectoriels.
- Documenter chaque méthode avec une docstring.

3.2 Splines

À faire : Implémenter la classe `CubicSpline` avec les méthodes suivantes :

- `__init__(self, x_points, y_points, boundary='natural')`
- `_compute_coefficients(self)`
- `_solve_tridiagonal(self, A, b)` (algorithme de Thomas)
- `evaluate(self, x_eval)`
- `derivative(self, x_eval, order=1)`
- `_find_interval(self, x)` (dichotomie)

À faire : Implémenter la classe `LinearSpline` avec les méthodes suivantes :

- `__init__(self, x_points, y_points)`
- `evaluate(self, x_eval)`
- `_find_interval(self, x)`

3.3 Méthodes d'intégration

À faire : Implémenter la classe `NewtonCotes` avec les méthodes statiques suivantes :

- `rectangle(f, a, b, n=1)`
- `trapezoidal(f, a, b, n=1)`
- `simpson(f, a, b, n=2)` (vérifier que n est pair)
- `simpson_38(f, a, b, n=3)` (bonus)

À faire : Implémenter la classe `AdaptiveIntegration` avec les méthodes suivantes :

- `__init__(self, tol=1e-6, max_depth=20)`
- `_simpson_rule(self, f, a, b)`
- `adaptive_simpson(self, f, a, b, tol=None, depth=0)`

À faire (bonus) : Implémenter la classe `GaussQuadrature` avec :

- `gauss_legendre_2(f, a, b)`
- `gauss_legendre_3(f, a, b)`

3.4 Applications

À faire : Implémenter la classe `CoolingProblem` avec :

— `__init__(self, t_data, T_data, T_ambient=20.0, h_coeff=50.0)`
— `temperature(self, t_eval)`
— `heat_loss_rate(self, t_eval)`
— `total_heat_loss(self, method='adaptive', n=100)`
— `exponential_model(self, t_eval, k)`
— `model_error(self, k)`
— `estimate_k(self, k_min=0.01, k_max=0.5, tol=1e-4)`

À faire : Implémenter la classe `FlowProblem` avec :

— `__init__(self, x_data, v_data, width_func=None)`
— `velocity(self, x_eval)`
— `local_flow_rate(self, x_eval)`
— `total_flow_rate(self, method='adaptive', n=100)`
— `acceleration(self, x_eval)`
— `work(self, mass=2.0)`

3.5 Visualisation

À faire : Implémenter la classe `Visualizer` avec les méthodes suivantes :

— `__init__(self, style='seaborn-v0_8-darkgrid', figsize=(10, 6))`
— `plot_interpolation_comparison(self, x_data, y_data, interpolators, x_fine, title)`
— `plot_runge_phenomenon(self, x_fine, y_true, interpolations)`
— `plot_convergence(self, n_values, errors, methods, title)`
— `plot_cooling_analysis(self, t_data, T_data, t_fine, T_interp, k_opt, T_model)`
— `plot_flow_analysis(self, x_data, v_data, x_fine, v_interp, w_function)`

3.6 Programme principal

À faire : Écrire un programme `main.py` qui :

1. Charge les données expérimentales.
2. Étudie le phénomène de Runge.
3. Analyse le problème de refroidissement.
4. Analyse le problème d'écoulement.
5. Étudie la convergence des méthodes d'intégration.
6. Affiche et sauvegarde les résultats.
7. Produit les graphiques demandés.

4 Travail d'analyse (à rédiger dans le rapport)

4.1 Analyse de l'interpolation

À faire : Répondre aux questions suivantes dans le rapport :

1. Quelle méthode (Lagrange, Newton, spline linéaire, spline cubique) donne les meilleurs résultats pour les données de refroidissement ? Justifier. Tracer les différentes interpolations sur un même graphique.
2. Pour la fonction $f(x) = \dfrac{1}{1+25x^2}$, comparer l'interpolation sur points équidistants et sur points de Tchebychev pour $n = 5, 10, 15, 20$. Expliquer ce que vous observez.
3. Pour les données de refroidissement, calculer la température à $t = 2,5$ s et $t = 7,3$ s avec chaque méthode. Commenter les différences.

4.2 Analyse de l'intégration

À faire : Répondre aux questions suivantes :

1. Pour la fonction test $f(x) = e^x$ sur $[0, 1]$, calculer l'erreur pour les méthodes du rectangle, des trapèzes et de Simpson pour $n = 2, 4, 8, 16, 32, 64, 128, 256$. Tracer les courbes d'erreur en échelle log-log. Déterminer l'ordre de convergence observé et le comparer à l'ordre théorique.
2. Comparer l'erreur et le nombre d'évaluations de la méthode adaptative avec la méthode de Simpson composée avec $n = 100$. Quel est l'avantage de la méthode adaptative ?
3. Pour le refroidissement, comparer les résultats obtenus avec les différentes méthodes d'intégration. Calculer le débit total pour l'écoulement avec chaque méthode.

4.3 Problème inverse

À faire : Répondre aux questions suivantes :

1. Pour le refroidissement, déterminer la valeur de k qui minimise :

$$E(k) = \int_0^{10} |T_{\text{exp}}(t) - T_{\text{modèle}}(t, k)|\, dt$$

   Tracer $E(k)$ en fonction de k sur l'intervalle $[0,01; 0,5]$.
2. Tracer sur un même graphique les données expérimentales, l'interpolation spline et le modèle exponentiel avec le k optimal. Calculer l'erreur maximale et l'erreur moyenne du modèle.

5 Livrables attendus

5.1 Code source

À fournir :
— L'ensemble du code source organisé de manière modulaire.
— Un fichier `requirements.txt` listant les dépendances.
— Un fichier `README.md` expliquant comment installer et exécuter le projet.
Critères d'évaluation :
— Le code doit être exécutable sans erreur.
— Le code doit respecter les conventions PEP 8.
— Chaque fonction/méthode doit être documentée (docstring).
— Le code doit être modulaire et bien structuré.

5.2 Rapport

À fournir : Un rapport de 8 à 10 pages comprenant :

1. **Introduction** (1 page) : contexte, problématique, objectifs.
2. **Partie théorique** (2 pages) : formules, méthodes, analyse théorique.
3. **Partie pratique** (2 pages) : description de l'implémentation, architecture logicielle, difficultés rencontrées.
4. **Résultats et analyse** (3 pages) : résultats, graphiques, tableaux, interprétation.
5. **Conclusion** (1 page) : bilan, enseignements, perspectives.

5.3 Soutenance

À présenter : Une soutenance orale de 10 à 15 minutes comprenant :
— Présentation du projet (5 min)
— Démonstration du code (5 min)
— Analyse des résultats (3 min)
— Questions-réponses (2 min)

6 Annexes

6.1 Annexe A : Données complémentaires

6.1.1 A.1 Fonction de Runge

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-5, 5]$$

6.1.2 A.2 Points de Tchebychev

$$x_k = \cos\left(\frac{2k-1}{2n}\pi\right), \quad k = 1, \ldots, n$$

Transformation sur $[a, b]$:

$$x_k^{(a,b)} = \frac{a+b}{2} + \frac{b-a}{2} x_k$$

6.1.3 A.3 Fonction test pour la convergence

$$f(x) = e^x, \quad \int_0^1 e^x \, dx = e - 1 \approx 1,718281828459045$$

6.2 Annexe B : Données expérimentales

Les données expérimentales sont fournies en section 1.1.

Bon courage et bon projet !